
ML–Driven Prediction and Ranking of High Energy Density Materials

Postgraduate Project Report

MTH697

by

Banoth Anand

Roll No - 228170281



DEPARTMENT OF MATHEMATICS AND STATISTICS
INDIAN INSTITUTE OF TECHNOLOGY KANPUR

June 2026

Acknowledgement

I would like to express my deepest gratitude to professor **Srinivas Dharavath** for his unwavering guidance, encouragement and invaluable mentorship throughout the completion of this project. I would also like to thank my department supervisor **Mrinmay Biswas** for always supporting and motivating me to conduct research.

Prof. Srinivas Dharavath

Associate Professor

CHM Department

Indian Institute of Technology Kanpur

Prof. Mrinmay Biswas

Assistant Professor

MTH Department

Indian Institute of Technology Kanpur

Declaration

I/We hereby declare that the work presented in the project report entitled "**ML–Driven Prediction and Ranking of High Energy Density Materials**" is written by me in my own words and contains my own or borrowed ideas. At places, where ideas and words are borrowed from other sources, proper references and acknowledgements, as applicable, have been provided. To the best of my knowledge this work does not emanate from or resemble work created by person(s) other than those mentioned and acknowledged herein.

Banoth Anand

Roll no- 228170281

Dual-MTH

Date: 06/04/2026

Abstract

High Energy Density Materials (HEDMs) are important in applications like propellants, explosives, and energy systems. Their performance depends on factors such as heat of formation, density, and detonation velocity. Traditionally, testing these materials is expensive, slow, and often dangerous. In this work, we present a machine learning-driven framework for predicting and screening HEDMs using molecular representations derived from SMILES strings.

We developed a feature engineering pipeline using RDKit to pull out physicochemical descriptors, structural features, and extended-connectivity (Morgan) fingerprints. We also added features specific to the domain, such as oxygen balance, nitrogen ratio, and nitro group count, to capture energy traits. We trained and optimized various regression models, including Linear Regression, Random Forest, Gradient Boosting, and XGBoost, through cross-validation to predict key properties: heat of formation, density, and detonation velocity.

We evaluated the models with standard metrics like MAE, RMSE, and R^2 . Ensemble methods showed better predictive performance. The framework also uses rule-based filtering and min-max normalization to find and rank promising HEDM candidates. Finally, we extended the system to screen unseen compounds, allowing for the quick discovery of high-potential materials.

This study shows how combining cheminformatics and machine learning can speed up the design and evaluation of energetic materials.

Contents

1	Introduction	1
2	Literature Review	3
3	Machine Learning Framework for HEDM Prediction and Ranking	5
3.1	Problem Formulation and Dataset Description	5
3.2	Data Preprocessing and Feature Engineering	6
3.2.1	Molecular Descriptors (RDKit)	7
3.2.2	Structural Features (Element Counts and Ratios)	7
3.2.3	Domain-Specific Features (Oxygen Balance and Nitro Groups)	8
3.2.4	Morgan Fingerprints	8
3.3	Model Development	9
3.3.1	Linear Regression Model	9
3.3.2	Random Forest Regressor	10
3.3.3	Gradient Boosting Regressor	10
3.3.4	XGBoost Regressor	11
3.4	Model Training and Hyperparameter Optimization	11
3.5	Model Evaluation Metrics and Results	12
4	Ranking Framework for High Energy Density Materials	14
4.1	Prediction of Target Properties (HOF, Density, VOD)	14
4.2	Rule-Based Qualification Criteria	15
4.2.1	Density Threshold	15
4.2.2	Oxygen Balance Constraint	15
4.2.3	Detonation Velocity Threshold	15
4.3	Normalization, Scoring, and Ranking of Candidate Materials	16
5	Prediction on Unseen Compounds	18
5.1	Input Data Handling and Feature Extraction	18
5.2	Property Prediction using Trained Models	18
5.3	Ranking and Analysis of New Compounds	19
6	Code Implementation	21
7	Visualization and Analysis	34
7.1	Distribution of Target Variables	34
7.2	Model Performance and Prediction Analysis	35
8	Conclusion and Future Work	36

Chapter 1

Introduction

High Energy Density Materials (HEDMs) are a crucial group of compounds commonly used in propellants, explosives, and energetic systems. The effectiveness of these materials mainly depends on their physicochemical properties, such as heat of formation (HOF), density, and detonation velocity (VOD), which are closely tied to their molecular structure. Estimating these properties accurately is important for designing energetic materials safely and effectively. However, traditional experimental methods are costly and time-consuming, and they also come with significant safety risks because of the hazardous nature of these compounds. As a result, there is an increasing need for reliable computational methods that can efficiently predict material properties and help discover promising candidates.

From a mathematical viewpoint, predicting material properties from molecular representations can be seen as a high-dimensional nonlinear regression problem. With a mapping from molecular space (represented by SMILES strings) to a feature space made up of descriptors and structural fingerprints, the goal is to learn functions that capture the relationships between these features and target properties. This naturally leads to using machine learning models, which can handle complex, nonlinear relationships in high-dimensional data.

In this project, we present a machine learning framework for predicting and ranking HEDMs using molecular descriptors and structural fingerprints obtained from cheminformatics tools. The process begins with thorough feature engineering that includes physicochemical descriptors, atom-level structural statistics, and specific quantities like oxygen balance and nitro group count. We enhance these features with high-dimensional Morgan fingerprints, which provide a detailed representation of molecular topology.

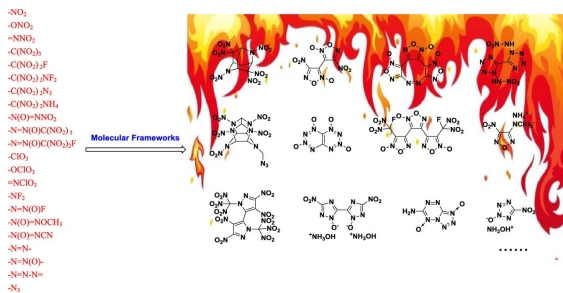


FIGURE 1.1: Molecular Architectures of Energetic Systems

We train and optimize several regression models, including Linear Regression, Random Forest, Gradient Boosting, and XGBoost, to predict the target properties. We evaluate the models using strict statistical metrics like Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). In addition to prediction, we introduce a structured ranking framework that com-

combines normalized property scores with specific constraints to find high-potential candidates.

This integrated method connects mathematical modeling, machine learning, and computational chemistry, offering a scalable and effective way to speed up the design and evaluation of high energy density materials.

Chapter 2

Literature Review

The challenge of predicting material properties from molecular structure sits at the crossroads of applied mathematics, statistical learning, and computational chemistry. Traditionally, estimating properties for energetic materials has depended on empirical correlations and quantum chemical simulations. Methods like density functional theory (DFT) offer accurate estimates, but they are costly in terms of computation and not suitable for large-scale screening. This has led to the creation of data-driven methods that estimate the relationships between molecular structure and macroscopic properties.

Mathematically, the goal is to learn a function

$$f : \mathbb{R}^d \rightarrow \mathbb{R} \tag{2.1}$$

that connects a high-dimensional feature vector describing a molecule to a single property, such as heat of formation or detonation velocity.

Classical linear regression models assume a linear relationship in the form

$$y = X\beta + \epsilon, \tag{2.2}$$

where $X \in \mathbb{R}^{n \times d}$ is the feature matrix, β are the model parameters, and ϵ is the error term. While these models are interpretable, they struggle to capture nonlinear interactions found in chemical systems.

To address these issues, tree-based ensemble methods have become popular. Random Forest (RF) models create a group of decision trees using bootstrap aggregation, which reduces variance and improves generalization. Gradient Boosting Machines (GBMs) further enhance prediction performance by sequentially minimizing a loss function using additive

modeling:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x), \quad (2.3)$$

where $h_m(x)$ is a weak learner and γ_m is the step size.

XGBoost, a refined version of gradient boosting, adds regularization and efficient computation, making it especially effective for high-dimensional structured datasets.

Feature representation is a key part of these models. In cheminformatics, molecular descriptors and fingerprints are commonly used to numerically represent chemical information. Extended-connectivity fingerprints (ECFP), also called Morgan fingerprints, represent molecules as binary vectors that capture local atomic environments. These representations can be seen as mappings from molecular graphs to high-dimensional vector spaces, relating to concepts in graph theory and functional approximation.

Recent research has shown the benefits of combining domain knowledge with machine learning features. For energetic materials, descriptors like oxygen balance, nitrogen content, and counts of functional groups (such as nitro groups) have been strongly linked to detonation performance. For example, oxygen balance is defined as:

$$OB = \frac{1600(n_O - 2n_C - \frac{n_H}{2})}{M}, \quad (2.4)$$

where n_C , n_H , n_O are the numbers of carbon, hydrogen, and oxygen atoms, and M is the molecular weight. Features grounded in physical properties improve both the interpretability and the accuracy of models.

In addition to prediction, multi-criteria decision-making and ranking methods have grown more important in materials discovery. These methods integrate multiple predicted properties into a single score through normalization and aggregation techniques, allowing for the identification of the best candidates based on given constraints. Mathematically, this can be viewed as creating a composite objective function across multiple outputs.

Despite advances, challenges persist in merging strong feature engineering, model selection, and domain-specific constraints into one cohesive framework. This project adds to the existing research by creating a complete pipeline that incorporates cheminformatics-based feature extraction, ensemble machine learning models, and a robust ranking mechanism. The approach prioritizes both predictive accuracy and practical usability, offering a methodologically sound way to accelerate the discovery of high energy density materials.

Chapter 3

Machine Learning Framework for HEDM Prediction and Ranking

3.1 Problem Formulation and Dataset Description

The prediction of physicochemical properties of High Energy Density Materials (HEDMs) can be formulated as a supervised learning problem in a high-dimensional feature space. Let each molecule be represented by a SMILES string s_i , which encodes its molecular structure. Through a feature extraction process, each molecule is mapped to a vector

$$x_i \in \mathbb{R}^d, \quad (3.1)$$

where d denotes the dimensionality of the feature space comprising molecular descriptors, structural features, and fingerprint representations.

The objective is to learn a set of functions

$$f_k : \mathbb{R}^d \rightarrow \mathbb{R}, \quad k \in \{1, 2, 3\}, \quad (3.2)$$

that map molecular features to target properties:

- Heat of Formation (HOF),
- Density,
- Detonation Velocity (VOD).

Given a dataset of n samples

$$\{(x_i, y_i^{(1)}, y_i^{(2)}, y_i^{(3)})\}_{i=1}^n, \quad (3.3)$$

the goal is to approximate the unknown mappings f_k such that prediction error is minimized under suitable loss functions.

The dataset used in this study, *det dataset 08-02-2022.xlsx*, is obtained from a publicly available repository of energetic materials. It consists of experimentally or computationally derived properties of known HEDMs. Each entry contains a molecular representation in the form of a SMILES string, along with key physicochemical properties including heat of formation (HOF), density, and detonation velocity (VOD).

This dataset provides a structured mapping between molecular structure and energetic performance, enabling the application of regression-based machine learning models. The presence of diverse molecular structures and varying energetic properties makes it suitable for learning complex nonlinear relationships.

The problem is further extended beyond prediction to a ranking task, where predicted properties are combined to identify high-performing candidate materials under domain-specific constraints.

3.2 Data Preprocessing and Feature Engineering

The raw dataset requires systematic preprocessing to ensure data quality and compatibility with machine learning models. The preprocessing pipeline involves validation, cleaning, and transformation of molecular representations into numerical feature vectors.

Initially, only the relevant columns—SMILES, HOF, Density, and VOD—are selected from the dataset. Entries with missing target values are removed to avoid inconsistencies during model training. Additionally, the validity of each SMILES string is verified using cheminformatics tools to ensure that only chemically valid molecular structures are retained.

Following data cleaning, the primary challenge lies in transforming symbolic molecular representations into numerical vectors suitable for regression models. This is achieved through a multi-level feature engineering approach, combining physicochemical descriptors, structural attributes, domain-specific chemical features, and high-dimensional fingerprint representations.

This hybrid feature construction strategy is motivated by the need to capture both global molecular properties and local structural patterns. The resulting feature matrix

$$X \in \mathbb{R}^{n \times d} \quad (3.4)$$

serves as the input for all subsequent machine learning models.

3.2.1 Molecular Descriptors (RDKit)

Molecular descriptors provide quantitative measures of physicochemical properties derived from molecular structure. These descriptors are computed using the RDKit library, which offers a wide range of chemically meaningful features.

In this study, key descriptors include molecular weight, topological polar surface area (TPSA), number of hydrogen bond donors, number of hydrogen bond acceptors, logarithm of the partition coefficient (LogP), and ring count.

Mathematically, these descriptors can be viewed as functions

$$\phi_j : \mathcal{M} \rightarrow \mathbb{R}, \quad (3.5)$$

where $\mathcal{M}$ denotes the space of molecular graphs. Each descriptor captures a specific aspect of molecular behavior, such as polarity, size, or structural complexity.

These features are particularly important as they provide low-dimensional, interpretable representations that correlate with macroscopic properties like density and stability. For instance, molecular weight influences density, while TPSA relates to intermolecular interactions.

3.2.2 Structural Features (Element Counts and Ratios)

In addition to standard descriptors, explicit structural features based on atomic composition are extracted. These include the number of carbon atoms (n_C), number of hydrogen atoms (n_H), number of nitrogen atoms (n_N), and number of oxygen atoms (n_O).

From these counts, derived quantities such as the nitrogen ratio are computed:

$$\text{Nitrogen Ratio} = \frac{n_N}{\text{Total Number of Atoms}}. \quad (3.6)$$

These features provide direct insight into the chemical composition of molecules. In the context of energetic materials, nitrogen-rich compounds are often associated with higher energy release due to the formation of stable N_2 gas during detonation.

Structurally, these features correspond to simple graph-theoretic properties of the molecular graph, where atoms are nodes and bonds are edges. Their inclusion enhances the model’s ability to capture compositional effects that are not fully represented by generic descriptors.

3.2.3 Domain-Specific Features (Oxygen Balance and Nitro Groups)

To incorporate domain knowledge specific to energetic materials, additional features are engineered based on established chemical principles.

One of the most important features is the oxygen balance (OB), defined as:

$$OB = \frac{1600 \left(n_O - 2n_C - \frac{n_H}{2} \right)}{M}, \quad (3.7)$$

where n_C , n_H , and n_O denote atomic counts and M is the molecular weight.

Oxygen balance quantifies the extent to which a molecule contains sufficient oxygen to fully oxidize carbon and hydrogen during detonation. Positive or near-zero values typically indicate efficient energy release, making this a critical parameter for HEDM evaluation.

Another key feature is the count of nitro groups ($-NO_2$), which are identified using substructure pattern matching. Nitro groups are known to significantly enhance energetic performance due to their high oxidizing potential.

These domain-specific features introduce physically meaningful constraints into the model, improving both predictive accuracy and interpretability.

3.2.4 Morgan Fingerprints

While descriptors and structural features capture global properties, they may fail to represent detailed local structural patterns. To address this, Morgan fingerprints—also known as Extended Connectivity Fingerprints (ECFP)—are used.

Morgan fingerprints encode molecular structures as high-dimensional binary vectors:

$$f \in \{0, 1\}^d, \quad (3.8)$$

where each bit corresponds to the presence or absence of a particular substructure within a specified radius.

The generation process involves iteratively updating atom identifiers based on their neighboring environments, effectively capturing local topology. In this work, fingerprints are generated with a fixed radius and dimensionality (e.g., 1024 bits), ensuring a consistent representation across all molecules.

From a mathematical perspective, this can be viewed as embedding molecular graphs into a high-dimensional feature space, enabling models to distinguish between subtle structural variations.

The combination of fingerprint features with descriptors and domain-specific attributes results in a comprehensive representation that balances interpretability and expressive power, making it well-suited for regression-based prediction of energetic material properties.

3.3 Model Development

The prediction of energetic material properties is inherently a nonlinear regression problem in a high-dimensional feature space. To approximate the mapping

$$f : \mathbb{R}^d \rightarrow \mathbb{R}, \quad (3.9)$$

multiple regression models with varying levels of complexity and expressiveness are considered.

The models selected in this study include Linear Regression (baseline linear model), Random Forest (bagging-based ensemble model), Gradient Boosting (additive boosting model), and XGBoost (regularized and optimized boosting model).

This selection ensures a comprehensive comparison between linear, ensemble, and advanced boosting-based approaches. Each model captures different aspects of the underlying data distribution, allowing for a robust evaluation of predictive performance.

3.3.1 Linear Regression Model

Linear Regression serves as a baseline model for comparison. It assumes a linear relationship between input features and the target variable, expressed as:

$$y = x^\top \beta + \epsilon, \quad (3.10)$$

where $\beta \in \mathbb{R}^d$ represents model parameters and ϵ denotes noise.

The parameters are estimated by minimizing the least squares objective:

$$\min_{\beta} \|X\beta - y\|_2^2. \quad (3.11)$$

While this model is computationally efficient and interpretable, it is limited in capturing nonlinear interactions between molecular features, which are crucial in chemical systems.

3.3.2 Random Forest Regressor

Random Forest is an ensemble learning method based on the principle of bootstrap aggregation (bagging). It constructs multiple decision trees using random subsets of data and features, and aggregates their predictions.

The final prediction is given by:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T h_t(x), \quad (3.12)$$

where h_t represents the t -th decision tree.

Random Forest reduces variance and improves generalization, making it suitable for high-dimensional datasets. It is particularly effective in handling nonlinear relationships and interactions between features without requiring explicit feature transformations.

3.3.3 Gradient Boosting Regressor

Gradient Boosting builds an additive model by sequentially fitting weak learners (typically decision trees) to the residuals of previous models.

At iteration m , the model is updated as:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x), \quad (3.13)$$

where h_m is the weak learner and γ_m is the step size.

The model minimizes a differentiable loss function using gradient descent in function space. This approach allows the model to focus on hard-to-predict samples, improving overall accuracy.

Gradient Boosting is well-suited for structured tabular data and provides strong predictive performance with relatively simple tuning.

3.3.4 XGBoost Regressor

XGBoost (Extreme Gradient Boosting) is an optimized implementation of gradient boosting that incorporates regularization, efficient computation, and advanced optimization techniques.

The objective function is given by:

$$L = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_k \Omega(h_k), \quad (3.14)$$

where ℓ is the loss function and Ω is a regularization term that penalizes model complexity.

XGBoost introduces regularization to prevent overfitting, parallel processing for computational efficiency, handling of missing values, as well as shrinkage and subsampling techniques.

These features make it highly effective for high-dimensional feature spaces such as those generated by molecular fingerprints.

3.4 Model Training and Hyperparameter Optimization

To ensure optimal performance, each model is trained using a systematic hyperparameter tuning strategy. The dataset is split into training and testing subsets, and feature scaling is applied using standardization.

Hyperparameter optimization is performed using grid search with cross-validation. The objective is to minimize the root mean squared error (RMSE) across validation folds.

The parameter grids used in this study are:

Random Forest:

- Number of estimators: 200
- Maximum depth: None, 10

Gradient Boosting:

- Number of estimators: 200
- Learning rate: 0.05, 0.1

XGBoost:

- Number of estimators: 300
- Maximum depth: 4, 6
- Learning rate: 0.05, 0.1
- Subsample: 0.8
- Column sample by tree: 0.8

Linear Regression does not require hyperparameter tuning.

Cross-validation (with $k = 5$ folds) is employed to ensure that the selected models generalize well to unseen data. The best-performing model for each target variable is selected based on evaluation metrics.

3.5 Model Evaluation Metrics and Results

The performance of the trained models is evaluated using standard regression metrics that quantify the accuracy of predictions. The primary metrics considered in this study are Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2).

The Mean Absolute Error is defined as:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (3.15)$$

which measures the average magnitude of prediction errors without considering their direction.

The Root Mean Squared Error is given by:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (3.16)$$

which penalizes larger errors more strongly due to the squared term, making it particularly suitable for scientific applications.

The coefficient of determination is defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad (3.17)$$

and measures the proportion of variance in the target variable explained by the model.

The models are evaluated separately for each of the three target properties: Heat of Formation (HOF), Density, and Detonation Velocity (VOD). The results demonstrate that different models perform optimally for different targets.

For the prediction of Heat of Formation (HOF), the XGBoost model achieved the best performance among all considered models, with a Mean Absolute Error of approximately 189.64 and a Root Mean Squared Error of approximately 252.61. The corresponding coefficient of determination was $R^2 \approx 0.728$, indicating strong predictive capability.

In the case of Density, the Gradient Boosting model exhibited superior performance, achieving an MAE of approximately 0.089 and an RMSE of approximately 0.118, with $R^2 \approx 0.525$. Similarly, for Detonation Velocity (VOD), Gradient Boosting again provided the best results, with an MAE of approximately 0.578 and an RMSE of approximately 0.850, and $R^2 \approx 0.303$.

Linear Regression performed poorly across all tasks, particularly in terms of both error metrics and R^2 . This observation indicates that the relationship between molecular features and energetic properties is highly nonlinear, and therefore requires more expressive models such as ensemble methods.

To systematically identify the best model for each target variable, a ranking strategy based on error metrics is employed. The models are ranked primarily using RMSE, with MAE serving as a secondary criterion to resolve ties. This corresponds to sorting the models in ascending order of RMSE, followed by MAE.

RMSE is chosen as the primary metric because it penalizes large errors, which is critical in scientific applications where such deviations can lead to incorrect conclusions. MAE is used complementarily to provide a balanced view less sensitive to outliers. R^2 is not used for model selection due to several reasons: it does not reflect absolute prediction error, can be misleading in low-variance datasets.

Based on the above evaluation and selection criteria, the final models chosen for subsequent analysis are as follows: XGBoost for Heat of Formation (HOF), and Gradient Boosting for both Density and Detonation Velocity (VOD). These models are used in the later stages of the framework for prediction, ranking, and evaluation of new candidate materials.

Chapter 4

Ranking Framework for High Energy Density Materials

While accurate prediction of energetic material properties is essential, practical applications require identifying compounds that simultaneously satisfy multiple performance criteria. This necessitates a systematic framework for evaluating and ranking candidate materials based on predicted properties and domain-specific constraints.

In this work, a structured ranking framework is proposed that integrates machine learning predictions with chemically motivated rules and a composite scoring mechanism. The framework operates in three stages: prediction of target properties, rule-based qualification, and normalization-based scoring and ranking.

4.1 Prediction of Target Properties (HOF, Density, VOD)

The first stage of the ranking framework involves predicting the key energetic properties of each compound using the trained machine learning models.

Given a dataset of molecules represented by feature vectors

$$\{x_1, x_2, \dots, x_n\}, \tag{4.1}$$

the predicted properties are obtained as:

$$\hat{y}_i^{(\text{HOF})} = f_{\text{HOF}}(x_i), \tag{4.2}$$

$$\hat{y}_i^{(\text{Density})} = f_{\text{Density}}(x_i), \tag{4.3}$$

$$\hat{y}_i^{(\text{VOD})} = f_{\text{VOD}}(x_i). \tag{4.4}$$

These predictions correspond to heat of formation (energy content), density (mass per unit volume), and detonation velocity (performance indicator).

These three properties collectively determine the energetic efficiency and practical usability of a material. However, optimizing a single property is insufficient, as real-world applications require a balance among all three.

4.2 Rule-Based Qualification Criteria

Before ranking, candidate materials are filtered using domain-specific constraints to ensure that only physically meaningful and practically viable compounds are considered.

A compound is classified as **HEDM-qualified** if it satisfies the following conditions:

4.2.1 Density Threshold

$$\hat{y}^{(\text{Density})} \geq 1.75 \text{ g/cm}^3 \quad (4.5)$$

High density is a critical requirement for energetic materials, as it directly influences detonation pressure and energy release.

4.2.2 Oxygen Balance Constraint

$$OB > 0 \quad (4.6)$$

Oxygen balance is defined as:

$$OB = \frac{1600 \left(n_O - 2n_C - \frac{n_H}{2} \right)}{M}, \quad (4.7)$$

where n_C , n_H , and n_O are atomic counts and M is the molecular weight.

A positive oxygen balance indicates that the molecule contains sufficient oxygen to fully oxidize its fuel components, leading to efficient energy release during detonation.

4.2.3 Detonation Velocity Threshold

$$\hat{y}^{(\text{VOD})} \geq 7000 \text{ m/s} \quad (4.8)$$

Detonation velocity is a key indicator of explosive performance. Higher values correspond to more powerful and efficient materials.

These rule-based constraints act as a filtering mechanism, ensuring that only compounds with physically meaningful and desirable characteristics are considered in the ranking process. Compounds failing to meet these criteria are deprioritized, regardless of their individual property values.

4.3 Normalization, Scoring, and Ranking of Candidate Materials

Following qualification, the remaining compounds are ranked using a composite scoring approach that combines multiple predicted properties into a single evaluative metric.

(i) Normalization of Predicted Properties

Since the predicted properties have different scales and units, min-max normalization is applied to each property:

$$\tilde{y}_i = \frac{y_i - y_{\min}}{y_{\max} - y_{\min}}, \quad (4.9)$$

where $y_{\min}$ and $y_{\max}$ denote the minimum and maximum values of the property across the dataset.

This transformation maps all values to the interval $[0, 1]$, enabling fair comparison across different properties.

(ii) Composite Scoring Function

A unified score is computed for each compound as the average of normalized properties:

$$\text{Score}_i = \frac{1}{3} \left(\tilde{y}_i^{(\text{HOF})} + \tilde{y}_i^{(\text{Density})} + \tilde{y}_i^{(\text{VOD})} \right). \quad (4.10)$$

This formulation assumes equal importance of all three properties, reflecting a balanced optimization strategy. From a mathematical perspective, this can be interpreted as constructing a linear scalarization of a multi-objective optimization problem.

(iii) Ranking Strategy

The final ranking is performed by sorting compounds based on two criteria:

- **Qualification Status (primary criterion):** Qualified compounds are ranked above non-qualified ones.

- **Composite Score (secondary criterion):** Within each group, compounds are ranked in descending order of the final score.

Formally, this corresponds to lexicographic ordering:

$$(\text{Qualified}_i, \text{Score}_i). \quad (4.11)$$

(iv) Interpretation and Significance

The proposed ranking framework provides a systematic and interpretable approach for identifying promising HEDM candidates. By integrating machine learning predictions, chemical domain knowledge, and multi-objective scoring, the framework bridges the gap between data-driven modeling and practical material selection.

Unlike purely predictive models, this approach enables decision-making by prioritizing compounds that satisfy both quantitative performance metrics and qualitative chemical constraints.

Chapter 5

Prediction on Unseen Compounds

Unlike the training phase, which relies on labeled data, this stage operates on a new dataset containing only molecular representations in the form of SMILES strings. The new compounds were obtained directly from laboratory synthesis with no previously recorded physicochemical properties. Their molecular structures were drawn using ChemDraw software, from which the corresponding SMILES strings were extracted and compiled into a structured Excel file, forming the new unseen compound dataset. The objective is to generate feature representations, apply the trained models to estimate target properties, and rank the compounds to identify the most promising HEDM candidates.

5.1 Input Data Handling and Feature Extraction

In this stage, the new dataset containing only SMILES strings of the laboratory-obtained compounds is loaded and validated using RDKit, and any chemically invalid entries are discarded. The feature extraction process is then performed using the same pipeline described in Section 3.2, and the resulting feature vectors are scaled using the StandardScaler fitted during training to ensure consistency. The extracted features are passed to the best saved models to predict the target properties.

5.2 Property Prediction using Trained Models

The prediction of target properties for unseen compounds is carried out using the final selected models obtained in Section 3.

This step follows the same methodology described in Section 4.1

5.3 Ranking and Analysis of New Compounds

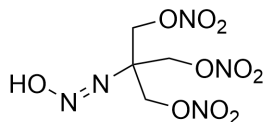
The ranking of unseen compounds is carried out using the same framework described in Section 4.3. Based on the predicted performance metrics, the compounds are ordered according to their suitability as high energy density materials.

From this ranking, the top three candidate compounds are identified for further analysis. These compounds demonstrate the most promising predicted values across key properties such as heat of formation, density, and detonation performance.

The top-ranked compounds SMILES are listed below:

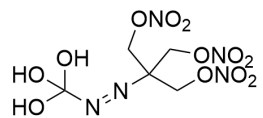
- **Rank 1:** ON=NC(CO[N+]([O-])=O)(CO[N+]([O-])=O)CO[N+]([O-])=O
- **Rank 2:** OC(O)(O)N=NC(O[N+]([O-])=O)(O[N+]([O-])=O)O[N+]([O-])=O
- **Rank 3:** OC(O)(O)N=NC(O)(O)O

Visualization of Top Candidates



2-((hydroxydiazanyl)-2-((nitrooxy)methyl)propane-1,3-diyl dinitrate

FIGURE 5.1: visualization of the Rank 1 HEDM Compound.



2-((nitrooxy)methyl)-2-((trihydroxymethyl)diazanyl)propane-1,3-diyl dinitrate

FIGURE 5.2: visualization of the Rank 2 HEDM Compound.

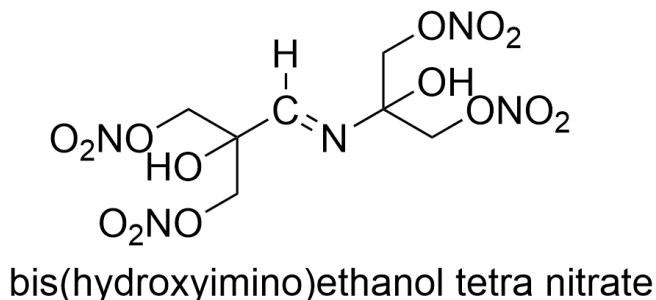


FIGURE 5.3: visualization of the Rank 3 HEDM Compound.

Notably, prior theoretical and experimental studies have independently identified a particular compound as a highly promising candidate for high energy density material applications, and efforts are currently underway to synthesize this compound in the laboratory. It is significant that the same compound has emerged as the top-ranked candidate among the set of 30 unseen compounds evaluated in this study. This agreement between experimental intuition and model prediction provides strong evidence that the machine learning models are well-trained and capable of accurately capturing the underlying relationships governing the target properties.

Analysis of Top Compounds

A detailed analysis of these top-ranked compounds is performed by examining both their predicted properties and structural characteristics. Particular attention is given to functional groups contributing to high energy release, molecular stability, and structural features influencing detonation performance. This analysis provides insights into structural patterns associated with superior energetic performance.

Chapter 6

Code Implementation

The complete implementation of the proposed framework is provided below.

```
1 # -*- coding: utf-8 -*-
2 """HEDM_ML_Project.ipynb
3
4 Automatically generated by Colab.
5
6 Original file is located at
7     https://colab.research.google.com/drive/1
8     cXX0negttfi3ZgnCP4gEsnhh19HpaQ4m
9 """
10 !pip install rdkit
11
12 import rdkit
13 print("RDKit version:", rdkit.__version__)
14
15 from rdkit import Chem
16 mol = Chem.MolFromSmiles('CCO') # ethanol
17 print(f"Molecule created: {Chem.MolToSmiles(mol)}")
18
19 !pip install openpyxl scikit-learn pandas numpy matplotlib seaborn
20     joblib xgboost
21
22 from rdkit import Chem
23 from rdkit.Chem import Descriptors
24
25 import pandas as pd
26 import numpy as np
27 import joblib
28 import os
```

```
28 import matplotlib.pyplot as plt
29 import seaborn as sns
30
31 from rdkit import Chem
32 from rdkit.Chem import Descriptors
33 from rdkit.Chem.rdFingerprintGenerator import GetMorganGenerator
34
35 from sklearn.model_selection import train_test_split, GridSearchCV
36 from sklearn.preprocessing import StandardScaler
37 from sklearn.metrics import mean_absolute_error, mean_squared_error
38     , r2_score
39
40 from sklearn.ensemble import RandomForestRegressor,
41     GradientBoostingRegressor
42 from sklearn.linear_model import LinearRegression
43 from xgboost import XGBRegressor
44
45 # =====
46 # STEP 1: Load Dataset
47 # =====
48
49 DATA_PATH = "det_dataset_08-02-2022.xlsx"
50 df = pd.read_excel(DATA_PATH)
51
52 df = df[['SMILES', 'HOF', 'Density', 'VODexp']]
53 df = df.dropna().reset_index(drop=True)
54 df.head()
55
56 # =====
57 # STEP 2: Cleaning & Preprocessing
58 # =====
59
60 def is_valid_smiles(smiles):
61     return Chem.MolFromSmiles(smiles) is not None
62
63 df = df.dropna()
64 df = df[df['SMILES'].apply(is_valid_smiles)].reset_index(drop=True)
65
66 print("Cleaned data rows:", df.shape[0])
67
68 # =====
69 # STEP 3: Feature Generation
70 # =====
```

```
70 morgan_gen = GetMorganGenerator(radius=2, fpSize=1024)
71
72 def oxygen_balance(mol):
73     formula = Descriptors.MolWt(mol)
74     nC = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'C')
75     nH = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'H')
76     nO = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'O')
77
78     if formula == 0:
79         return 0
80
81     OB = (1600 * (nO - 2*nC - nH/2)) / formula
82     return OB
83
84 def count_nitro_groups(mol):
85     nitro = Chem.MolFromSmarts('[N+](=O)[O-]')
86     return len(mol.GetSubstructMatches(nitro))
87
88 def generate_features(smiles):
89     mol = Chem.MolFromSmiles(smiles)
90     if mol is None:
91         return None
92
93     # Basic descriptors
94     descriptors = [
95         Descriptors.MolWt(mol),
96         Descriptors.TPSA(mol),
97         Descriptors.NumHDonors(mol),
98         Descriptors.NumHAcceptors(mol),
99         Descriptors.MolLogP(mol),
100        Descriptors.RingCount(mol)
101    ]
102
103    # Element counts
104    nC = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'C')
105    nH = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'H')
106    nN = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'N')
107    nO = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'O')
108
109    total_atoms = mol.GetNumAtoms()
110    nitrogen_ratio = nN / total_atoms if total_atoms > 0 else 0
111
112    OB = oxygen_balance(mol)
113    nitro_count = count_nitro_groups(mol)
```

```
114
115     structural_features = [
116         nC, nH, nN, nO,
117         nitrogen_ratio,
118         OB,
119         nitro_count
120     ]
121
122     fp = np.array(morgan_gen.GetFingerprint(mol))
123
124     return np.concatenate([descriptors, structural_features, fp])
125
126 X = np.vstack(df['SMILES'].apply(generate_features))
127
128 y_hof = df['HOF'].values
129 y_density = df['Density'].values
130 y_vod = df['VODexp'].values
131
132 print("Feature matrix shape:", X.shape)
133
134 # =====
135 # STEP 4: Train Test Split & Scaling
136 # =====
137
138 X_train, X_test, y_hof_train, y_hof_test, \
139 y_density_train, y_density_test, \
140 y_vod_train, y_vod_test = train_test_split(
141     X, y_hof, y_density, y_vod,
142     test_size=0.2,
143     random_state=42
144 )
145
146 scaler = StandardScaler()
147 X_train = scaler.fit_transform(X_train)
148 X_test = scaler.transform(X_test)
149
150 # =====
151 # STEP 5: Model Definitions
152 # =====
153
154 models = {
155     "RF": RandomForestRegressor(random_state=42),
156     "GB": GradientBoostingRegressor(random_state=42),
157     "LR": LinearRegression(),
```

```
158     "XGB": XGBRegressor(  
159         objective="reg:squarederror",  
160         random_state=42,  
161         n_jobs=-1  
162     )  
163 }  
164  
165 param_grids = {  
166     "RF": {"n_estimators": [200], "max_depth": [None, 10]},  
167     "GB": {"n_estimators": [200], "learning_rate": [0.05, 0.1]},  
168     "LR": {},  
169     "XGB": {  
170         "n_estimators": [300],  
171         "max_depth": [4, 6],  
172         "learning_rate": [0.05, 0.1],  
173         "subsample": [0.8],  
174         "colsample_bytree": [0.8]  
175     }  
176 }  
177  
178 # =====  
179 # STEP 6: Model Training  
180 # =====  
181  
182 def train_models(X, y):  
183     trained = {}  
184  
185     for name, model in models.items():  
186         if name == "LR":  
187             model.fit(X, y)  
188             trained[name] = model  
189         else:  
190             grid = GridSearchCV(  
191                 model,  
192                 param_grids[name],  
193                 cv=5,  
194                 scoring="neg_root_mean_squared_error",  
195                 n_jobs=-1  
196             )  
197             grid.fit(X, y)  
198             trained[name] = grid.best_estimator_  
199  
200     return trained  
201
```

```

202 hof_models = train_models(X_train, y_hof_train)
203 density_models = train_models(X_train, y_density_train)
204 vod_models = train_models(X_train, y_vod_train)
205
206 # =====
207 # STEP 7: Model Evaluation
208 # =====
209
210 def evaluate(models, X_test, y_test):
211     scores = {}
212     for name, model in models.items():
213         preds = model.predict
214         scores[name] = {
215             "MAE": mean_absolute_error(y_test, preds),
216             "RMSE": np.sqrt(mean_squared_error(y_test, preds)),
217             "R2": r2_score(y_test, preds)
218         }
219     return pd.DataFrame(scores).T
220
221 hof_scores = evaluate(hof_models, X_test, y_hof_test)
222 density_scores = evaluate(density_models, X_test, y_density_test)
223 vod_scores = evaluate(vod_models, X_test, y_vod_test)
224
225 hof_scores, density_scores, vod_scores
226
227 # =====
228 # STEP 8: Save Best Models
229 # =====
230
231 def select_best_model(scores_df, trained_models):
232     # Sort by RMSE first, then MAE
233     ranked = scores_df.sort_values(by=["RMSE", "MAE"], ascending=[
234         True, True])
235
236     best_model_name = ranked.index[0]
237     best_model = trained_models[best_model_name]
238
239     print(f"Best model selected: {best_model_name}")
240     print(ranked.loc[best_model_name])
241
242     return best_model_name, best_model
243
244 os.makedirs("models", exist_ok=True)

```

```
245
246 best_hof_name, best_hof = select_best_model(hof_scores, hof_models)
247 best_density_name, best_density = select_best_model(density_scores,
248     density_models)
249
250 joblib.dump(best_hof, "models/hof_model.pkl")
251 joblib.dump(best_density, "models/density_model.pkl")
252 joblib.dump(best_vod, "models/vod_model.pkl")
253 joblib.dump(scaler, "models/scaler.pkl")
254
255 print("\nFinal selected models:")
256 print(f"HOF           {best_hof_name}")
257 print(f"Density       {best_density_name}")
258 print(f"VOD           {best_vod_name}")
259
260 # =====
261 # STEP 9: Screening & Normalized Ranking
262 # =====
263
264 df['Pred_HOF'] = hof_models["XGB"].predict(scaler.transform(X))
265 df['Pred_Density'] = density_models["XGB"].predict(scaler.transform
266     (X))
267
268 df['Pred_VODexp'] = vod_models["XGB"].predict(scaler.transform(X))
269
270 # Recalculate oxygen balance for rules
271 df['OxygenBalance'] = df['SMILES'].apply(
272     lambda x: oxygen_balance(Chem.MolFromSmiles(x))
273 )
274
275 # Rule Check
276 df['HEDM_Qualified'] = (
277     (df['Pred_Density'] >= 1.75) &
278     (df['OxygenBalance'] > 0) &
279     (df['Pred_VODexp'] >= 7000)
280 )
281
282 # Normalized ranking
283 for col in ['Pred_HOF', 'Pred_Density', 'Pred_VODexp']:
284     df[col + "_norm"] = (df[col] - df[col].min()) / (df[col].max()
285         - df[col].min())
286
287 df['Final_Score'] = (
288     df['Pred_HOF_norm'] +
```



```
286     df['Pred_Density_norm'] +
287     df['Pred_VODexp_norm']
288 ) / 3
289
290 ranked_df = df.sort_values(
291     by=['HEDM_Qualified', 'Final_Score'],
292     ascending=False
293 )
294
295 ranked_df.to_csv("ranked_HEDM_compounds.csv", index=False)
296
297 plt.figure(figsize=(15,4))
298
299 plt.subplot(1,3,1)
300 sns.histplot(df['HOF'], kde=True)
301 plt.title("Heat of Formation Distribution")
302
303 plt.subplot(1,3,2)
304 sns.histplot(df['Density'], kde=True)
305 plt.title("Density Distribution")
306
307 plt.subplot(1,3,3)
308 sns.histplot(df['VODexp'], kde=True)
309 plt.title("Detonation Velocity Distribution")
310
311 plt.tight_layout()
312 plt.show()
313
314 def plot_predicted_vs_actual(y_true, y_pred, title):
315     plt.figure(figsize=(5,5))
316     plt.scatter(y_true, y_pred, alpha=0.6)
317     plt.plot([y_true.min(), y_true.max()],
318             [y_true.min(), y_true.max()], 'r--')
319     plt.xlabel("Actual")
320     plt.ylabel("Predicted")
321     plt.title(title)
322     plt.show()
323
324 # HOF
325 plot_predicted_vs_actual(y_hof_test, best_hof.predict(X_test),
326                         "HOF: Predicted vs Actual")
327
328 # Density
```

```
329 plot_predicted_vs_actual(y_density_test, best_density.predict(
    X_test),
330                             "Density: Predicted vs Actual")
331
332 # VOD
333 plot_predicted_vs_actual(y_vod_test, best_vod.predict(X_test),
334                             "VOD: Predicted vs Actual")
335
336 # =====
337 # STEP 10: Prediction on New / Unseen Compounds
338 # =====
339
340 def predict_unseen(smiles_list):
341     feats = np.vstack([generate_features(s) for s in smiles_list])
342     feats_scaled = scaler.transform(feats)
343
344     return pd.DataFrame({
345         "SMILES": smiles_list,
346         "Pred_HOF": best_hof.predict(feats_scaled),
347         "Pred_Density": best_density.predict(feats_scaled),
348         "Pred_VODexp": best_vod.predict(feats_scaled)
349     })
350
351 unseen = ["C1(=NN=N1)[N+](=O)[O-]"]
352 predict_unseen(unseen)
353
354 # =====
355 # STEP 11: Predict + Rank Compounds from Uploaded CSV
356 # =====
357
358 New_compounds_CSV_PATH = "New_Compound_SMILES(1).csv"
359 unseen_df = pd.read_csv(New_compounds_CSV_PATH)
360
361 assert "SMILES" in unseen_df.columns, "CSV must contain a 'SMILES'
    column"
362
363 # =====
364 # Load trained models & scaler
365 # =====
366
367 best_hof = joblib.load("models/hof_model.pkl")
368 best_density = joblib.load("models/density_model.pkl")
369 best_vod = joblib.load("models/vod_model.pkl")
370 scaler = joblib.load("models/scaler.pkl")
```

```
371
372 # =====
373 # Feature generation
374 # =====
375 morgan_gen = GetMorganGenerator(radius=2, fpSize=1024)
376
377 def oxygen_balance(mol):
378     mol_wt = Descriptors.MolWt(mol)
379     nC = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'C')
380     nH = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'H')
381     nO = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'O')
382
383     if mol_wt == 0:
384         return 0
385
386     OB = (1600 * (nO - 2*nC - nH/2)) / mol_wt
387     return OB
388
389 def count_nitro_groups(mol):
390     nitro = Chem.MolFromSmarts('[N+](=O)[O-]')
391     return len(mol.GetSubstructMatches(nitro))
392
393 def generate_features(smiles):
394     mol = Chem.MolFromSmiles(smiles)
395     if mol is None:
396         return None
397
398     # Basic descriptors
399     descriptors = [
400         Descriptors.MolWt(mol),
401         Descriptors.TPSA(mol),
402         Descriptors.NumHDonors(mol),
403         Descriptors.NumHAcceptors(mol),
404         Descriptors.MolLogP(mol),
405         Descriptors.RingCount(mol)
406     ]
407
408     # Element counts
409     nC = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'C')
410     nH = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'H')
411     nN = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'N')
412     nO = sum(1 for a in mol.GetAtoms() if a.GetSymbol() == 'O')
413
414     total_atoms = mol.GetNumAtoms()
```

```
415     nitrogen_ratio = nN / total_atoms if total_atoms > 0 else 0
416
417     OB = oxygen_balance(mol)
418     nitro_count = count_nitro_groups(mol)
419
420     structural_features = [
421         nC, nH, nN, nO,
422         nitrogen_ratio,
423         OB,
424         nitro_count
425     ]
426
427     fp = np.array(morgan_gen.GetFingerprint(mol))
428
429     return np.concatenate([descriptors, structural_features, fp])
430
431 features = []
432 valid_indices = []
433
434 for idx, smiles in enumerate(unseen_df["SMILES"]):
435     feat = generate_features(smiles)
436     if feat is not None:
437         features.append(feat)
438         valid_indices.append(idx)
439
440 features = np.vstack(features)
441 features_scaled = scaler.transform(features)
442
443 unseen_df = unseen_df.iloc[valid_indices].reset_index(drop=True)
444
445 # =====
446 # Predictions
447 # =====
448 unseen_df["Pred_HOF"] = best_hof.predict(features_scaled)
449 unseen_df["Pred_Density"] = best_density.predict(features_scaled)
450 unseen_df["Pred_VODexp"] = best_vod.predict(features_scaled)
451
452
453 unseen_df["OxygenBalance"] = unseen_df["SMILES"].apply(
454     lambda x: oxygen_balance(Chem.MolFromSmiles(x))
455 )
456
457 def contains_CHNO(smiles):
458     mol = Chem.MolFromSmiles(smiles)
```

```

459     mol = Chem.AddHs(mol)
460     atoms = set(a.GetSymbol() for a in mol.GetAtoms())
461     return all(x in atoms for x in ["C", "H", "N", "O"])
462
463 unseen_df["HEDM_Qualified"] = (
464     (unseen_df["Pred_Density"] >= 1.60) &
465     (unseen_df["OxygenBalance"] > 0) &
466     (unseen_df["Pred_VODexp"] >= 7) &
467     (unseen_df["SMILES"].apply(contains_CHNO))
468 )
469
470 '''
471 Density unit    g/cm^3
472 OxygenBalance unit %
473 VODexp unit    km/s
474 '''
475 # =====
476 # Min Max Normalization (same as Step 9)
477 # =====
478
479 for col in ["Pred_HOF", "Pred_Density", "Pred_VODexp"]:
480     unseen_df[col + "_norm"] = (
481         (unseen_df[col] - unseen_df[col].min()) /
482         (unseen_df[col].max() - unseen_df[col].min())
483     )
484
485 unseen_df["Final_Score"] = (
486     unseen_df["Pred_HOF_norm"] +
487     unseen_df["Pred_Density_norm"] +
488     unseen_df["Pred_VODexp_norm"]
489 ) / 3
490
491
492 # =====
493 # Ranking
494 # =====
495
496 ranked_unseen_df = unseen_df.sort_values(
497     by=["HEDM_Qualified", "Final_Score"],
498     ascending=False
499 )
500
501 OUTPUT_PATH = "ranked_unseen_HEDM_compounds.csv"

```

```
502 ranked_unseen_df.to_csv(OUTPUT_PATH, index=True, index_label="
    Original_Index")
503
504 print("    Ranked file saved as:", OUTPUT_PATH)
505 ranked_unseen_df.head(30)
```

LISTING 6.1: Complete Implementation of the HEDM ML Code

Chapter 7

Visualization and Analysis

7.1 Distribution of Target Variables

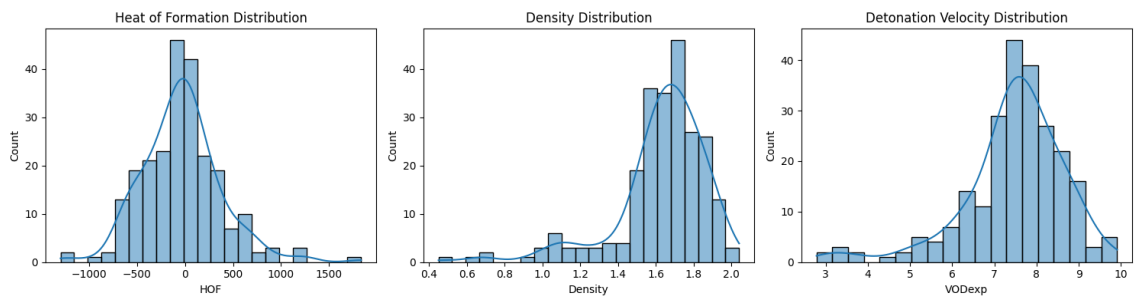


FIGURE 7.1: Target Variables Distribution Plot

The above histograms illustrate the distribution of the three target variables: Heat of Formation (HOF), Density, and Detonation Velocity (VOD). These plots provide insight into the data spread and reveal that the distributions are moderately skewed, indicating variability and non-uniformity in the dataset.

7.2 Model Performance and Prediction Analysis

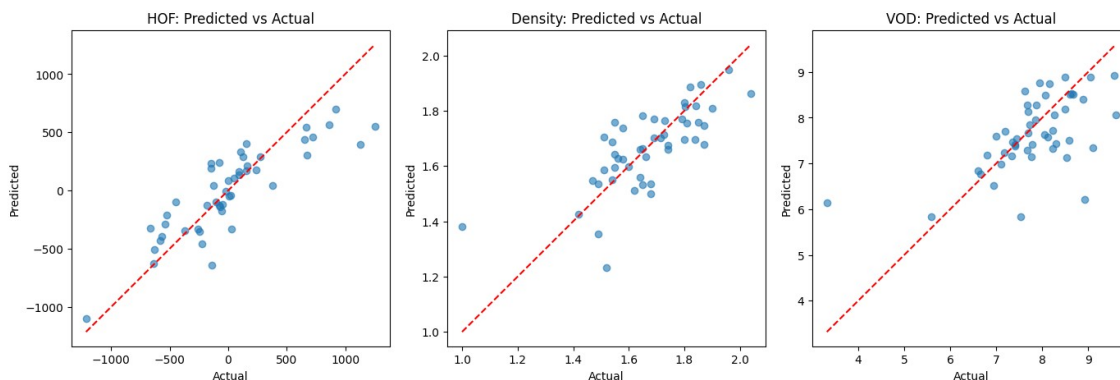


FIGURE 7.2: Target variables Pred vs Actual

The scatter plots show the relationship between actual and predicted values for each target variable. The red dashed line represents the ideal case where predicted values perfectly match the actual values.

For HOF, Density, and VOD, the predictions generally follow the diagonal trend, indicating that the models capture the underlying patterns reasonably well, although some deviations and spread are observed due to model limitations and data complexity.

Initially, Linear Regression was considered based on the apparent linear trend in the predicted vs actual plots. However, it performed poorly because the relationship between molecular features and target properties is inherently nonlinear. Linear models fail to capture complex interactions between features, whereas ensemble methods like Gradient Boosting and XGBoost are better suited for such nonlinear dependencies.

Chapter 8

Conclusion and Future Work

This study presents a machine learning-based framework for the prediction and ranking of High Energy Density Materials (HEDMs) using molecular descriptors and structural fingerprints. The results demonstrate that ensemble models, particularly XGBoost and Gradient Boosting, effectively capture the nonlinear relationships between molecular features and target properties such as heat of formation, density, and detonation velocity.

The proposed ranking framework integrates model predictions with domain-specific constraints to identify promising candidate materials in a systematic manner. Application to unseen compounds further highlights the generalization capability and practical relevance of the approach.

Future work may focus on incorporating larger and more diverse datasets, exploring graph-based learning methods, and developing more advanced multi-objective optimization strategies to further improve predictive performance and material discovery.

Bibliography

- [1] N. V. Muravyev, D. R. Wozniak, and D. G. Piercey, “Progress and performance of energetic materials: open dataset, tool, and implications for synthesis,” *Journal of Materials Chemistry A*, vol. 10, no. 21, pp. 11054–11073, 2022.
- [2] K. A. Monogarov, K. A. Monogarov, and A. N. Pivkina, “Energy transferred to energetic materials during impact test: a more realistic approach,” *Defence Technology*, vol. 19, pp. 1–10, 2023.
- [3] N. V. Muravyev, “Energetic materials dataset and prediction tools.” <http://chemphys.space/shiny/dataset/>, 2022. Accessed: 2026.